

Entrega: A entrega da resolução é realizada na página da disciplina no Moodle juntando, num único ficheiro compactado, o **código fonte, o Makefile**. Existirão aulas práticas para a realização parcial deste trabalho com entrega na respetiva aula.

Objetivo: Familiarização com as ferramentas de desenvolvimento C em ambiente UNIX/LINUX (gcc, make, gdb) e de um ambiente integrado de desenvolvimento (VsCode). Conceção de programas com acesso aos argumentos da linha de comando de um programa, acesso às variáveis de ambiente de um sistema UNIX e estudo das chamadas de sistema relacionadas com a criação de processos (fork, getpid, getppid, wait, waitpid), execução de programas (família de funções exec), mecanismo de comunicação *pipe* (pipe) e redireccionamento de I/O (dup, dup2); Consolidação da programação ao nível de sistema.

Livro: A resolução deste trabalho pressupõe a leitura dos capítulos 1 a 6 do livro R. Arpaci-Dusseau, A. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, November, 2023 (Version 1.10).

I. Realize os seguintes exercícios

Durante a realização dos exercícios propostos utilize, no terminal, o comando `man` de forma a esclarecer dúvidas sobre as chamadas de sistema e as funções C, como por exemplo, quais os seus argumentos, os valores de retorno e como verificar as situações de erro. Para simplificar a verificação de erros sugere-se a utilização da função :

```
void fatal_system_error (const char *errorMsg) {
    perror(errorMsg);
    exit(EXIT_FAILURE);
}
```

Para a resolução de cada questão comece por criar uma pasta contendo os ficheiros C com a resolução do exercício e um ficheiro Makefile (Incluindo, entre outras, as regras **all**, e **clean**) que permita a compilação da solução.

Na sua resolução considere o tratamento de erros das chamadas de sistema utilizadas.

I. Realize os seguintes programas

1. Crie um programa que execute simultaneamente (em paralelo) os comandos `/bin/date` e `/bin/ping -c 4 www.google.com`. O programa deve esperar pelo fim da execução dos dois programas indicando o estado de terminação de cada um deles (chamadas de sistema a usar: fork, waitpid, família de funções exec).
2. Realize um programa (`count_words`) que recebe através da linha de argumentos o nome de um ficheiro e imprima no standard de output o número de palavras desse ficheiro. A determinação do número de palavras é realizada executando o programa externo `wc` com a opção `-w` (chamadas de sistema a usar: fork, pipe, dup/dup2, exec, waitpid). O programa `count_words` deve receber o número de palavras do programa `wc` e depois escrever no standard output o número de palavras.

II. Processamento utilizando múltiplos processos

3. Pretende-se um programa que dado um vetor determine o maior, o menor e a soma de todos os seus elementos. Junto do enunciado é fornecido um programa (`vector-seq.c`) que determina a informação pretendida num vetor com o limite máximo indicado nos argumentos da linha de comando. Este programa utiliza uma abordagem sequencial e apresenta-se, de seguida, um exemplo da sua utilização para uma dimensão de 1000 valores:

```
$ ./vector-seq 1000000000
Creating a vector of 1000000000 (1000.00 MB; 1.00 GB) values
This will require approximately 2000.00 MB (2.00 GB) of memory
smaller is -32768
bigger is 32767
The sum is 303681303
```

Numa primeira fase e utilizando o código da Figura 1, avalie o tempo de processamento sequencial para o vetor com a dimensão de 1000000000 valores

```
#define _XOPEN_SOURCE 600 // for clock_gettime
#include <stdio.h>
#include <time.h>

int main (void) {
    struct timespec t_start;
    struct timespec t_end;

    // Get the start time
    clock_gettime(CLOCK_MONOTONIC, &t_start);

    // code to be timed goes here
    ...

    // Get the end time
    clock_gettime(CLOCK_MONOTONIC, &t_end);

    // Calculate the elapsed time in seconds
    double elapsed_time = (t_end.tv_sec - t_start.tv_sec) + (t_end.tv_nsec - t_start.tv_nsec) * 1e-9;

    // Print the elapsed time in seconds and microseconds
    printf("Total elapsed time = %9.6lfs (%9.6lfus)\n", elapsed_time,
        elapsed_time * 1e6);

    return 0;
}
```

Figura 1 – Avaliação do tempo de execução de um troço de código

Com o objetivo de reduzir o tempo necessário, na determinação da informação pretendida, desenvolva uma nova versão do programa que distribui o processamento por múltiplos processos como ilustrado na Figura 2.

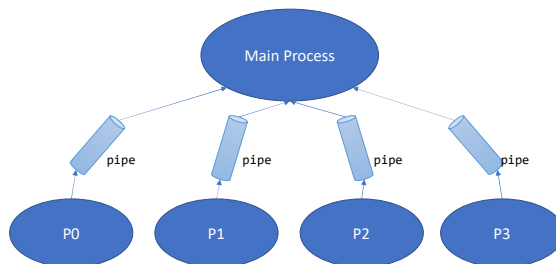


Figura 2 - Arquitetura geral da solução baseada em múltiplos processos

A estratégia a seguir é a seguinte:

- A nova versão do programa (`vector-seq-processes`) recebe mais um argumento, da linha de comandos, que indica o número de processos auxiliares a utilizar, por exemplo, para serem utilizados dois processos o programa deve ser executado da seguinte forma:

```
$> ./ vector-seq-processes 1000000000 2
```

- O processo principal divide o processamento do vetor em intervalos e envia-os aos processos filhos, i.e., se utilizarmos dois processos nos cálculos para um vetor de dimensão 1000, teríamos que o primeiro processo realiza os cálculos no intervalo entre 0 e 499 e o segundo processo entre 500 e 999. Neste caso deve tirar partido do facto da chamada de sistema `fork()` criar um processo através da clonagem do processo pai incluindo o seu espaço de endereçamento.
- Os processos filhos, no fim do processamento, enviam os resultados ao processo principal através do mecanismo `pipe` para que este agregue os resultados e os apresente ao utilizador.

Avalie os tempos de execução entre a versão sequencial e a versão baseada em múltiplos processos variando a dimensão do vetor e o número de processos.

Notas

- **As funções deste trabalho devem ser desenvolvidas recorrendo às chamadas de sistema estudadas: `open`, `read`, `write`, `close`, `fork`, `wait`, `exec`, `pipe` e `dup`, não usando as funções `system`, `popen`, etc.**
- **A solução de cada exercício tem de conter, obrigatoriamente, um Makefile** com as regras para a geração dos executáveis e uma regra para a eliminação dos ficheiros desnecessários da pasta do projeto.
- Esta proposta de trabalho deixa em aberto algumas questões que constituem opções a serem tomadas pelos alunos e que serão alvo de apreciação na avaliação do trabalho.
- Os testes realizados na verificação da correta implementação do trabalho fazem parte da avaliação.
- Valorize o seu trabalho tornando o código robusto através da validação das operações, deteção de erros, fecho dos descritores de ficheiros abertos, eliminação de processos *zombie*, etc.
- Nesta fase não se pretende um relatório formal, mas apenas uma pequena descrição da arquitetura das soluções e das opções realizadas.

III. Indique se as seguintes afirmações são verdadeiras (V) ou falsas (F)

4. Na avaliação a resposta certa será cotada com 100%, resposta errada -25%

a.

Nos sistemas operativos multiprogramados quando um processo realiza uma operação de espera de um determinado tempo, e.g. <code>sleep(1)</code> , o SO escolhe outro processo para continuar a executar-se.	
Nos sistemas operativos multiprogramados se um processo executar este ciclo infinito <pre>while (true) ;</pre> o SO suspende a execução desse processo para dar oportunidade dos outros processos se executarem.	
CPU <i>idle</i> é o tempo de processador não utilizado para executar código dos processos ou do próprio SO.	
Os sistemas operativos multi-utilizadores seguem as mesmas regras dos sistemas multiprogramados adicionado um tempo máximo que um processo pode utilizar um processador.	

b. Considere o programa seguinte.

<pre>01 int main() 02 { 04 fork(); 05 fork(); 06 fork(); 07 fork(); 08 fork(); 09 printf("Finishing\n"); 10 return 0; 11 } 12 }</pre>	A execução deste código irá originar além do processo principal mais 3 processos.	
	A execução deste código irá originar além do processo principal mais 7 processos.	
	Neste código enquanto o processo pai executa a linha 10 pode verificar-se processos zombies (<i>defuncts</i>) através da utilização do comando <code>ps</code> .	
	A linha 10 neste código é executada apenas pelo processo pai.	

C.

Se o processo pai termina antes do processo filho, o processo filho aborta a sua execução	
Se o processo pai termina antes do processo filho, o processo filho fica no estado zombie	
Se o processo filho termina antes do processo pai, o processo filho fica no estado zombie.	
Se o processo filho termina antes do processo pai, o processo filho fica órfão.	

d. Considere o programa seguinte e responda com verdadeiro (V) ou falso (F) às seguintes afirmações:

<pre>int main () { for (int i = 0; i < 10; ++i) { execlp("date", "date", NULL); sleep(1); } return 0; }</pre>	Este programa executa o programa date, para imprimir a data e hora no <i>standard de output</i> , em intervalos de 1 segundo durante 10 segundos.	
	São criados 10 processos auxiliares para executarem o programa date ficando esses processos no estado zombie.	
	Este código apenas executa o programa date 1 única vez.	
	Este código deveria esperar pelos processos filho através da utilização da chamada de sistema waitpid().	

Bom trabalho,
Nuno Oliveira